

Ferramentas adicionais

Na aula de hoje, veremos algumas ferramentas adicionais do Git, ainda assim, super úteis e muito utilizadas no laboratório e no desenvolvimento em robótica.

Na aula de hoje, veremos:

- Como dizer ao Git para ignorar arquivos e pastas específicos
- Como adicionar outros repositórios dentro do nosso
- Um novo parâmetro para o `git commit`
- Um novo comando

Git ignore

Um ponto importante é que o Git e o GitHub foram criados para armazenar código. No entanto, ao longo de um projeto, nossas pastas podem conter outros tipos de arquivos, como binários compilados em C++, pastas de log de uma execução específica, pastas de dados, entre outros.

Esses arquivos não devem fazer parte do repositório no GitHub e, portanto, não devem ser enviados. Mas como evitar isso?

A solução mais simples seria simplesmente não adicioná-los ao executar o `git add`. Porém, conforme o projeto cresce, começam a surgir muitos arquivos compilados e temporários, e o `git status` passa a ficar poluído com vários itens que não deveriam ser versionados. Nesse cenário, o ideal é ter uma configuração que informe ao Git quais arquivos ele não deve acompanhar.

É para isso que existe o `.gitignore`. Esse arquivo informa ao Git quais arquivos e/ou pastas devem ser ignorados no processo de versionamento.

O `.gitignore` nada mais é do que um arquivo de texto que fica na raiz do repositório e contém os caminhos relativos, a partir da raiz do projeto, para as pastas e arquivos que devem ser ignorados pelo Git.

Como criar um ignore

Veja o exemplo:

```
build/  
*.log  
dados/
```

Nesse exemplo, o git vai ignorar as pastas `build/`, `dados/` e também todos os arquivos terminados em `.log`. Existem diferentes formas de escrever e configurar como ignorar as pastas. Segue a tabela abaixo:

Padrão no <code>.gitignore</code>	O que faz	Exemplo prático
-----------------------------------	-----------	-----------------

Padrão no .gitignore	O que faz	Exemplo prático
arquivo.txt	Ignora um arquivo específico em qualquer lugar do projeto	Ignora qualquer arquivo.txt
/arquivo.txt	Ignora um arquivo específico apenas na raiz	Ignora apenas o arquivo.txt da raiz
*.log	Ignora todos os arquivos com determinada extensão	Ignora todos os arquivos .log
dados/	Ignora uma pasta chamada dados em qualquer lugar	Ignora qualquer diretório chamado dados
/dados/	Ignora a pasta dados apenas na raiz	Ignora somente /dados/ da raiz
dados/*	Ignora apenas o conteúdo da pasta dados , mas não a pasta em si	Mantém a pasta vazia versionada
**/temp/	Ignora pastas temp em qualquer nível do projeto	Funciona mesmo em subpastas profundas
!arquivo.txt	Remove a regra de ignore para um arquivo específico	Permite versionar arquivo.txt mesmo após um *.txt
dados/*.csv	Ignora apenas arquivos .csv dentro de dados	Outros tipos dentro de dados não são ignorados
*.o	Ignora arquivos objeto (comum em C/C++)	Muito usado em projetos de C e C++, como ROS por exemplo

Exemplos

```
*/build/  
dataset/*  
.vscode/
```

Ignora todas as pasta **build**, ignora todos os arquivos dentro da pasta **dataset** e ignora a pasta **.vscode**

Git submodules

O Git submodule é um recurso que permite adicionar um repositório Git dentro de outro repositório, como se fosse uma dependência externa controlada por versão. Em vez de copiar e colar o código de outro projeto para dentro do seu, você pode vinculá-lo como um submódulo, mantendo a conexão com o repositório original. Isso é muito útil quando queremos reutilizar bibliotecas, frameworks ou componentes desenvolvidos separadamente, algo bastante comum em projetos maiores e no desenvolvimento em robótica.

Adicionando um submodules

O Git submodule permite adicionar um repositório dentro de outro, mantendo a referência a um commit específico do repositório externo.

```
git submodule add <url-do-repositorio> caminho/destino
```

```
git submodule add https://github.com/exemplo/biblioteca.git libs/biblioteca
```

Isso irá:

- Clonar o repositório dentro da pasta especificada
- Criar (ou atualizar) o arquivo .gitmodules
- Registrar o submódulo no seu repositório

Depois disso é necessário fazer o commit:

```
git add .gitmodules libs/biblioteca  
git commit -m "Adicionando submódulo biblioteca"
```

Clonando um repositório que já possui submódulos

Ao clonar um projeto que usa submódulos:

```
git clone <url>
```

Depois é preciso iniciar e atualizar os submódulos:

```
git submodule init  
git submodule update
```

Ou de uma forma mais prática:

```
git clone --recursive <url>
```

Escolhendo um commit em específico

Para falar qual commit em específico o submodule deve ficar, primeiro você precisa adicionar o submodules seguindo os passos anteriores e depois precisa entrar na pasta do submodules:

```
cd libs/biblioteca
```

Mudar para o commit em específico utilizando o hash dele:

```
git checkout <hash-do-commit>
```

Nesse momento, volte para o repositório principal e registre a mudança

```
cd ../../  
git add libs/biblioteca  
git commit -m "Fixando submódulo biblioteca no commit a1b2c3d4"
```

Exemplo

Quando você adiciona um submódulo, o Git cria automaticamente o arquivo `.gitmodules`, que fica na raiz do projeto. Um exemplo:

```
[submodule "libs/biblioteca"]  
  path = libs/biblioteca  
  url = https://github.com/exemplo/biblioteca.git
```

- submodule "libs/biblioteca": Nome lógico do submódulo
- path: Caminho onde ele está dentro do projeto
- url: Repositório remoto original

Git cherry-pick

O comando `git cherry-pick` permite pegar um commit específico de outra branch e aplicá-lo na branch atual, sem precisar fazer merge completo.

Ele é muito útil quando:

- Você fez uma correção em uma branch e quer levar só aquela correção para outra
- Precisa aplicar um hotfix na main que já foi feito em uma branch de desenvolvimento
- Quer reaproveitar uma alteração isolada

Em vez de trazer toda a branch, você escolhe apenas um commit específico (pelo hash) e o Git reaplica aquela mudança na branch atual.

Como usar:

Descubra a hash do commit através do `git log` e vá para a branch a qual deseja aplicar a mudança:

```
git checkout main
```

E execute o cherry-pick:

```
git cherry-pick a1b2c3d4
```

O Git criará um novo commit na branch atual, com as mesmas alterações.

Se der conflito, resolva manualmente e faça:

```
git add .  
git cherry-pick --continue
```

E se quiser cancelar:

```
git cherry-pick --abort
```